

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.5

Dictionaries

Day 5 — looking things up by name instead of by position



Boston University

90 min

Concepts + live code

30 min

Practice notebook

1

Structure, used everywhere

Prepared & presented by **Srinivasa Sai Chava**

Before we start — five questions from Day 4

1 How many items does `marks[1:4]` give?

→ Three — the stop value is excluded.

2 What is `nums` after `nums = nums.sort()`?

→ None. In-place methods return nothing.

3 Difference between `sort()` and `sorted()`?

→ `sort()` changes the list; `sorted()` returns a new one.

4 Why is `b = a` not a copy?

→ Second name, same object. Use `a.copy()`.

5 How do you write an empty set?

→ `set()` — because `{}` is an empty dictionary.

Question 5 is the door into today: those curly braces belong to the dictionary.

How today's two hours are structured

1

The basics

Creating, accessing, KeyError and .get()

25 min

2

Iterating

keys, values, items — and what 'in' checks

20 min

3

Methods & patterns

Counting, grouping, merging

25 min

4

Nested dictionaries

Real-world shapes, JSON, safe access

15 min

5

Mini build

A gradebook from raw records

5 min

6

Practice notebook

You code, I walk around

30 min

Section 3's counting and grouping patterns are the two you will reuse most often all year.

By the end of today you will be able to

1

Build

Create dictionaries and add, update and remove entries

2

Read safely

Fetch a value without crashing when the key is missing

3

Iterate

Loop over keys, values, or both at once

4

Apply

Use the counting and grouping patterns on real, messy data

Why this is the most important structure in the module

A row of a spreadsheet is a dictionary. Data returned by any web API is a dictionary. A pandas DataFrame is built out of them, and every model you configure later takes its settings as one. Lists were the warm-up — this is the structure real data actually arrives in.

1

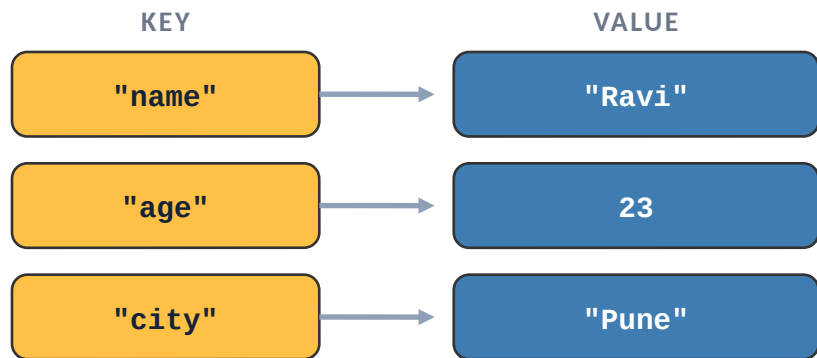
The Basics

Keys, values, and safe access · 25 minutes

- Key-value pairs
- Creating
- Accessing
- KeyError
- .get()
- Add / update / delete

A dictionary maps a key to a value

Look things up by NAME, not by position



Keys are unique. Values can be anything, and can repeat.

Compare with a list

```
# list - by position
student = ["Ravi", 23, "Pune"]
student[1]      # 23, but of what?
```

```
# dict - by name
student = {"name": "Ravi",
          "age": 23,
          "city": "Pune"}
student["age"]  # 23, unmistakably
```

The real difference is not syntax — it is that the data explains itself

With a list you must remember that position 1 holds the age. With a dictionary the data says so. Six months from now, reading your own code, that difference is the difference between understanding it and guessing.

Creating a dictionary and reading from it

Four ways to create one

```
marks = {"Python": 88, "Stats": 71}

empty = {} # empty dict
d = dict(name="Ravi", age=23) # keyword style
d = dict([("a", 1), ("b", 2)]) # from pairs
d = dict.fromkeys(["a", "b"], 0) # Day 4!
```

A missing key is a `KeyError`

```
marks = {"Python": 88, "Stats": 71}

marks["Python"] -> 88

marks["ML"]
-> KeyError: 'ML'

# It stops your program. Not a warning.
```

`.get()` is the safe way to read

```
marks.get("ML") -> None no crash
marks.get("ML", 0) -> 0 your default
marks.get("Python", 0) -> 88

"ML" in marks -> False check first
```

Use square brackets when the key must be there and its absence is a genuine bug you want to hear about. Use `.get()` with a default when the key is genuinely optional — reading a config setting, or a field a web API may not have sent. Choosing between them deliberately is what separates code that fails loudly from code that fails at 2am.

Adding, updating and removing entries

One syntax does both add and update

```
marks = {"Python": 88}

marks["Stats"] = 71    # key is new -> ADDS
marks["Python"] = 92   # key exists -> UPDATES

print(marks)
# {'Python': 92, 'Stats': 71}
```

⚠ There is no warning on overwrite

```
d = {"a": 1}
d["a"] = 2      # the 1 is simply gone

# To add ONLY if missing:
d.setdefault("a", 99) # a stays 2
d.setdefault("b", 99) # b becomes 99
```

Removing	What it does	If the key is missing
<code>del d[key]</code>	delete the entry	KeyError
<code>d.pop(key)</code>	delete it and return the value	KeyError
<code>d.pop(key, None)</code>	delete it, or give the default	no error
<code>d.clear()</code>	empty the whole dictionary	—



Iterating

Looping over keys, values and pairs · 20 minutes

- `keys()`
- `values()`
- `items()`
- What 'in' checks
- Order

Looping over a dictionary

Looping directly gives you the KEYS

```
marks = {"Python": 88, "Stats": 71}

for subject in marks:           # keys
    print(subject)

for m in marks.values():        # values
    print(m)

for subject, m in marks.items(): # BOTH
    print(subject, m)
```

'in' checks KEYS, never values

```
"Python" in marks      -> True
88 in marks             -> False !
88 in marks.values()    -> True
```

items() is the one you will use most

```
for subject, m in marks.items():
    status = "pass" if m >= 40 else "fail"
    print(f"{subject:10} {m:3} {status}")
```

Each item is a (key, value) tuple, and the loop unpacks it — Day 4's tuple unpacking, doing real work. It is the same shape as `enumerate()` and `zip()` from Day 2.

Dictionaries keep insertion order

Since Python 3.7, looping a dictionary gives keys back in the order you added them. Older tutorials will tell you dictionaries are unordered — that advice is out of date, and it is why `dict.fromkeys()` could deduplicate while keeping order yesterday.

3

Methods & Patterns

The two idioms you will reuse all year · 25 minutes

- Method reference
- Counting
- Grouping
- Merging

Dictionary methods worth knowing

Method	What it does	Example
<code>get(k, default)</code>	read safely, never raises	<code>d.get("ML", 0)</code>
<code>keys()</code>	a view of all keys	<code>list(d.keys())</code>
<code>values()</code>	a view of all values	<code>sum(d.values())</code>
<code>items()</code>	a view of (key, value) pairs	<code>for k, v in d.items()</code>
<code>setdefault(k, v)</code>	read k, inserting v first if absent	<code>d.setdefault("x", [])</code>
<code>update(other)</code>	merge another dict in, overwriting	<code>d.update({"a": 1})</code>
<code>pop(k, default)</code>	remove and return the value	<code>d.pop("a", None)</code>
<code>copy()</code>	a shallow copy — Day 4's lesson	<code>backup = d.copy()</code>

keys(), values() and items() return VIEWS, not lists. A view stays connected to the dictionary — add an entry and the view shows it. Wrap it in `list()` if you need a fixed snapshot, especially before modifying the dictionary you are looping over.

The counting pattern

How many times does each item appear?

```
words = ["a", "b", "a", "c", "a"]

counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1

print(counts)
# {'a': 3, 'b': 1, 'c': 1}
```

Why `.get(w, 0)` is the whole trick

```
counts[w] = counts[w] + 1
    -> KeyError on the FIRST time we
        see any new word
```

The first time a word appears there is nothing to add to. `.get(w, 0)` supplies a starting value of zero, so the very first count becomes 1. Without it you would need an if/else on every pass to check whether the key exists yet.

Python has a built-in for this — but write the loop first

```
from collections import Counter
Counter(words)
# Counter({'a': 3, 'b': 1, 'c': 1})
```

Counter does the same job in one line, and you should use it in real work. But write the manual loop at least a dozen times first — the pattern of “look up with a default, then update” appears everywhere, and Counter only solves this one instance of it.

The grouping pattern, and merging dictionaries

Collect items under a shared key

```
records = [("maths", "Ravi"), ("cs", "Sara"),
           ("maths", "Amit")]

groups = {}
for subject, name in records:
    groups.setdefault(subject, []).append(name)

print(groups)
# {'maths': ['Ravi', 'Amit'], 'cs': ['Sara']}
```

Merging two dictionaries

```
a.update(b)           # changes a in place
merged = {**a, **b}    # a new dict
merged = a | b         # Python 3.9+, cleanest
```

Reading that middle line

- `setdefault(subject, [])` returns the list for that subject
- If the subject is new, it inserts an empty list first, then returns it
- Either way you get a list back — so `.append()` always works
- One line replaces a four-line if/else

 **On a clash, the right-hand side wins**

`{"a": 1} | {"a": 2}` gives `{"a": 2}`. Merging defaults with user settings? Put the user's dictionary second.

4

Nested Dictionaries

The shape real data arrives in • 15 minutes

- Dicts inside dicts
- Lists of dicts
- Safe deep access
- Key rules

Nested dictionaries — how real data looks

A dictionary of dictionaries

```
students = {  
    "ravi": {"age": 23, "marks": [88, 71]},  
    "sara": {"age": 22, "marks": [91, 84]},  
}  
  
students["ravi"]["age"]      -> 23  
students["ravi"]["marks"][0] -> 88  
  
# Read left to right: one step per bracket.
```

A list of dictionaries

```
rows = [  
    {"name": "Ravi", "mark": 88},  
    {"name": "Sara", "mark": 91},  
]  
  
for r in rows:  
    print(r["name"], r["mark"])
```

This exact shape is what a CSV becomes, what a web API returns, and what pandas turns into a DataFrame.

Deep access fails deep — chain .get() to stay safe

```
students["neha"]["age"]      -> KeyError: 'neha'  
students.get("neha", {}).get("age") -> None
```

The empty {} default is the trick: if “neha” is missing you get an empty dictionary back, and calling .get() on that returns None instead of crashing.

What can be a key?

Keys must be immutable

```
d = {"name": 1}      # str    ok
d = {42: 1}          # int    ok
d = {(3, 7): 1}      # tuple  ok
d = {True: 1}        # bool   ok

d = {[1, 2]: 1}
-> TypeError: unhashable type: 'list'
```

Why lists are refused

A dictionary finds a value by computing a number from the key. If the key could change afterwards, that number would no longer match and the value would be lost inside the dictionary.

So Python refuses anything that can change: lists, sets and dictionaries. This is the practical reason tuples exist — and why a tuple containing a list is also refused.

Values have no such rule

```
d = {"marks": [88, 71, 64],      # a list
      "meta": {"term": 2},        # a dict
      "tags": {"cs", "maths"}}    # a set
```

A value can be absolutely anything, including another dictionary — which is exactly what makes the nested structures on the previous slide possible. Only the keys are constrained.

Debug corner — eight dictionary bugs

You wrote	What happens	Fix
<code>d["missing"]</code>	KeyError, program stops	<code>d.get("missing", default)</code>
<code>counts[w] = counts[w] + 1</code>	KeyError on the first sighting	<code>counts.get(w, 0) + 1</code>
<code>88 in marks</code>	False — 'in' checks keys	<code>88 in marks.values()</code>
<code>d = {[1,2]: "x"}</code>	TypeError: unhashable type	use a tuple as the key
<code>empty = {}</code>	a dict, not a set	<code>set()</code> if you wanted a set
<code>for k in d: del d[k]</code>	RuntimeError: changed size	<code>for k in list(d):</code>
<code>b = a</code>	both names change together	<code>b = a.copy()</code>
<code>d["a"]["b"]</code>	KeyError deep inside	<code>d.get("a", {}).get("b")</code>

Rows 1, 2 and 8 are all the same fix: reach for `.get()` whenever a key might not be there. All eight are in today's practice notebook.

Putting it together — a gradebook from raw records

```
records = [
    ("ravi", "python", 88), ("sara", "python", 91),
    ("ravi", "stats", 71),  ("amit", "python", 64),
    ("sara", "stats", 84),  ("ravi", "ml", 90),
]

def build_gradebook(rows):
    """Group (student, subject, mark) rows by student."""
    book = {}
    for name, subject, mark in rows:      # unpack the tuple
        book.setdefault(name, {})[subject] = mark
    return book

book = build_gradebook(records)

for name, subjects in book.items():
    marks = subjects.values()
    avg   = sum(marks) / len(marks)
    print(f"{name:6} {len(marks)} subjects   avg {avg:.1f}")

print("Ravi's stats mark:", book["ravi"].get("stats"))
print("Amit's ml mark   :", book["amit"].get("ml", "not taken"))
```



Nested dict

student → subject → mark



setdefault

creates the inner dict on demand



Tuple unpacking

three names from one row



items()

loop over names and marks



.values()

feed straight into sum()



.get() default

“not taken” instead of a crash

Six flat rows become a structure you can query by name — the same reshaping pandas does for you later.

Practice session — 30 minutes



Predict the output

KeyError, .get(), 'in', ordering

8 questions



Spot & fix the bug

Missing keys, counting, mutation while looping

4 questions



Write the code

Counting, grouping, inverting, filtering, merging

5 tasks

Your notebook

[Day5_Practice_Questions.ipynb](#)

- Print the dict after each step
- Ask: could this key be missing?
- Raise a hand — I am walking around
- Solutions shared after class

Most of today's errors are one word long: KeyError. Whenever a key might not be there, reach for .get().

Recap

- 1 A dictionary maps a unique key to a value.
- 2 `d[key]` raises `KeyError`; `d.get(key, default)` never does.
- 3 `d[key] = value` adds when new, overwrites when not.
- 4 Looping a dict gives you the KEYS.
- 5 `items()` gives (key, value) pairs — use it most.
- 6 `'in'` checks keys. Use `.values()` to search values.
- 7 Counting: `counts[k] = counts.get(k, 0) + 1`
- 8 Grouping: `d.setdefault(k, []).append(v)`
- 9 Merging: the right-hand side wins on a clash.
- 10 Keys must be immutable; values can be anything.
- 11 Dictionaries keep insertion order (Python 3.7+).
- 12 Never add or delete while looping — loop over `list(d)`.

Homework

Count the letters in a sentence · Invert a dictionary so values become keys · Extend the gradebook with a `top_student()` function

Next class — Topic 1.6

Comprehensions: one-line list, dict and set comprehensions, with conditions